

Программа для определения типа ориентированного графа	
Внутренняя спецификация	
Студент	Иванов Д.В.
Преподаватель	Якимов Г.А.
Сдано	

1. Назначение

Программа предназначена для анализа ориентированного графа на предмет того, является ли он деревом. Если не является, можно ли преобразовать его к дереву путём удаления некоторого набора дуг

2. Описание структур данных

//возможные виды графов в терминах задачи

Перечисление graphType содержит значения:

tree, convertibleToTree, notConvertibleToTree

//возможные виды ошибок

Перечисление ErrorType содержит значения:

inFileNotExist, createOutFileFail, edgeNumberError, verticesNumberError, defenitionLineSyntaxError, innerDescriptionSyntaxError, endLineSyntaxError

//структура для списка смежности графа

структура adjacencyList содержит поля:

std::vector<std::vector<int>> neighbours – описание дуг

adjacencyList * subtract(adjacencyList & subtrahend); - метод для

вычитания

`bool isEmpty()` - метод, возвращает `true`, если список пустой, `false` в ином случае

`bool isEqualTo(const adjacencyList & otherList)` – сравнение двух списков без учёта порядка описания дуг

`int countOfVertices` – кол-во вершин

Класс `Error` содержит поля:

`ErrorType type` – тип ошибки

`std::string errorTagName` – строка, хранящая имя неизвестного тега

`std::string errorInputFileWay` – строка пути, откуда не удалось прочитать файл

`std::string errorOutputFileWay` – строка пути, куда не удалось записать файл

`std::string generateErrorMessage()` - метод, возвращающий сообщение об ошибке

Класс `DirGraph` содержит поля:

`//набор дуг в виде списка смежности`

`adjacencyList edges`

`//конструктор`

`DirGraph()`

`//Деструктор`

`~DirGraph()`

`//метод добавления дуги в граф (return успешно ли)`

`bool addEdge(int fromVertex, int toVertex)` — метод для добавления дуги в граф

//метод извлечь набор дуг графа (список смежности)

adjacencyList * getEdges() - вернуть список смежности для операций над
ним

//метод для определения существования дуги между двумя вершинами

bool hasEdge(int fromVertex, int toVertex)

// вернуть количество вершин в графе

int getVertexCount() - вернуть количество вершин графа

// сравнение графов на равенство

bool dirGraphIsEqual(const DirGraph & otherGraph)

//создать остовное дерево графа с корнем в вершине rootVertex

DirGraph * generateSpanningTree(int rootVertex);

3. Описание алгоритмов функций

Главная функция программы:

int main(int argc, char* argv[]);

Входные данные: argc - количество переданных аргументов командной строки
argv[] - аргументы командной строки: argv[1] - путь к проектному файлу,
argv[2] - путь к выходной директории

Входные аргументы командной строки

Выходные данные: 0 – функция завершилась успешно

Алгоритм функции:

1. Считать из параметров командной строки путь к входному файлу

2. Считать из параметров командной строки путь для выходного файла
3. Если удалось открыть входной файл и ошибок не было обнаружено
 - 3.1 Считать данные из файла в граф
(readTextFromFile+parseGraphFromText)
4. Иначе если удалось открыть файл и ошибки были обнаружены
 - 4.1 Вернуть все обнаруженные ошибки
5. Иначе
 - 5.1 Вернуть ошибку о неудаче открытия файла
6. Закрыть файл
7. Если удалось считать данные и сформировать выходной файл без ошибок
 - 7.1 Определить тип входного графа (getGraphType)
 - 7.2 Занести в выходной файл результат (generateOutputFile)
8. Иначе
 - 8.1 Вернуть в консоль информацию об ошибках

Функция `DirGraph * DirGraph::generateSpanningTree(int rootVertex)`

Алгоритм:

1. Создать пустой граф с таким же набором вершин
2. Инициализировать массив посещённых вершин.
3. Считать все вершины кроме корня не посещёнными
4. Инициализировать очередь вершин с непосещёнными соседями корнем дерева
5. Пока в очереди есть вершины
 - 5.1 Извлечь вершину-источник из очереди
 - 5.2 Для всех непосещённых соседей вершины-источника
 - 5.2.1 Добавить дугу от источника к текущей вершине в граф
 - 5.2.2 Пометить текущую вершину посещённой
 - 5.2.3 Добавить текущую вершину в очередь
- 6 Вернуть полученный граф

//Функция чтения файла в обычный текст

```
bool readTextFromFile (std::vector<std::string> & fileText, std::string &
fileName, std::vector<Error> errorVector);
```

Входные данные — fileText – контейнер для записи текста; fileName – имя файла с текстом; errorsVector – контейнер для получившихся ошибок.

Выходные данные — fileText – контейнер для записи текста; errorsVector – контейнер для получившихся ошибок

Алгоритм функции:

1. Пытаемся открыть входной файл (std::ifstream inputFile(fileName);)
2. Если успешно открыли
 - 2.1 Пока в файле есть строки
 - 2.1.1 Считать следующую строку
 - 2.1.2 Записать строку в текст
3. Иначе
 - 3.1 Заносим ошибку чтения файла в вектор ошибок
 - 3.2 Вернуть false
4. Для каждой строки с конца пока не нашли непустую строку
Убрать строку из вектора
5. Вернуть true

//Парсинг графа из текста (вектора строк)

```
DirGraph * parseGraphFromText(std::vector<std::string> & fileText, int *
inDegrees[1000], std::vector<Error> & errorsVector);
```

Входные данные:

fileText – текстовое описание графа, int * inDegrees – массив полустепеней захода, errorsVector – вектор для ошибок

Выходные данные:

errorsVector – вектор для ошибок, return – ссылка на созданный граф

Алгоритм функции:

1. Создаём пустой граф
2. Извлекаем токены первой строки
3. Если набор токенов не соответствует шаблону
 - 3.1 Добавить в вектор ошибок defenitionLineSyntaxError,
4. Извлекаем токены из последней строки
5. Если набор токенов не соответствует шаблону (одионочная скобка «}»)
 - 5.1 Занести в вектор ошибок ошибку endLineSyntaxError
6. Для каждой строки, кроме первой и последней
 - 6.1 Извлекаем токены по разделителю «→» (extractTokensFromString)
 - 6.2 Если строка соответствует шаблону описания дуги
 - 6.2.1 Добавить в граф соответствующую дугу
 - 6.2.1 Инкрементировать степень захода вершины — приёмника
 - 6.3 Иначе если строка не соответствует описанию шаблона вершины
 - 6.3.1 Добавить в вектор ошибок innerDescriptionSyntaxError

//Функция для определения типа графа

```
graphType findGraphType(const DirGraph & graph, adjacencyList & edgeDifference, int inDegrees[1000]);
```

Входные данные: graph – входной граф

Выходные данные: return – тип графа; adjacencyList & edgeDifference – разность наборов дуг между графом и его остовным деревом

Алгоритм функции:

1. Найти количество потенциальных корней остовного дерева
2. Если количество потенциальных корней графа > 1
 - 2.1 Считаём граф неприводимым к дереву
3. Иначе Если количество потенциальных корней графа равно 1
 - 3.1 считать корнем остовного дерева вершину-потенциальный корень
 - 3.2 Построить остовное дерево
 - 3.3 Если полученное дерево охватывает все вершины
 - 3.3.1 Если разность дуг исходного и остовного графа пуста
 - 3.3.1.1 Считаём исходный граф деревом
 - 3.3.2 Иначе
 - 3.3.2.1 Считаём исходный граф приводимым к дереву
 - 3.4 Иначе
 - 3.4.1 Считаём граф неприводимым к дереву
4. Иначе
 - 4.1 Инициализируем массив опробованных вершин
 - 4.2 Инициализируем стек потенциальных корней первой вершиной
 - 4.3 Отмечаем первую вершину опробованной
 - 4.2 Пока стек непустой
 - 4.2.1 Извлечь вершину из стека
 - 4.2.2 Отметить извлечённую вершину опробованной
 - 4.2.3 Найти неопробованных родителей вершины
 - 4.2.4 Если родитель один
 - 4.2.4.1 заносим родителя в стек
 - 4.2.5 Иначе Если родителей > 1
 - 4.2.5.1 Занести родителей в стек в обратном порядке
 - 4.2.6 Иначе если родителей нет
 - 4.2.6.1 Построить остовное дерево из текущей вершины
 - 4.2.6.2 Если полученное дерево охватывает все вершины

4.2.6.2.1 Если разность дуг дерева и графа пустая

4.2.6.2.1.1 Считаем граф деревом

4.2.6.2.2 Иначе

4.2.6.2.2.1 Считаем граф приводимым

4.3 Считаем граф неприводимым

5. Вернуть тип графа

```
bool extractTokensFromString(std::string & str, std::string & delimiter,  
std::vector<std::string> & tokens);
```

- делит строку на токены по заданному разделителю, заносит их в вектор

Входные данные:

str – строка для разбиения на токены, delimiter – разделитель, tokens – вектор строк для участков строк между разделителями (токенов)

Выходные данные:

tokens - вектор строк для участков строк между разделителями (токенов)

Алгоритм функции:

1. Найти первый разделитель

2. Если разделитель пустой

2.1 вернуть false

3. Пока не дошли до конца строки

3.1 Занести в вектор подстроку между последней позицией указателя и текущей

3.2 Сдвинуть указатель последнего разделителями

3.3 Найти следующий разделитель

4. Вернуть true

//функция создания выходного файла


```
int generateOutputFile(graphType type, std::vector<std::string> &
inputFilePlainText, adjacencyList & edgesToDelete, std::string & outputFileName,
std::vector<Error> & errorsVector);
```

-сформировать выходной файл, если успешно — вернуть 0

Входные данные:

type – тип исходного графа

inputFilePlainText – считанный из входного файла текст

edgesToDelete – дуги, которые необходимо удалить

Выходные данные:

Создаётся файл с именем outputFileName

return – true, если успешно создан файл, false иначе

Алгоритм функции:

1. Инициализируем текст выходного файла
2. Создаём выходной файл по указанному пути
3. Если успешно
 - 3.1 копируем в текст выходного файла строку дефиниции графа
 - 3.2 Вставляем строку с описанием вершины результата анализа
 - 3.3 Копируем строки внутреннего описания графа из входного файла в выходной текст
 - 3.4 Если тип входного графа — приводимый к дереву
 - 3.4.1 Для каждой строки выходного текста
 - 3.4.1.1 Если текущая строка — описание дуги, подлежащей удалению
 - 3.4.1.1.1 Дополнить строку указанием красного цвета в скобках
4. Иначе
 - 4.1 Занести в вектор ошибок createOutFileFail

//функция вывода ошибок в консоль

void printErrorsMessages(std::vector<Error> & errorsVector)

Входные данные: errorsVector – вектор с ошибками

Выходные данные: вывод в консоль сообщений об ошибках

Алгоритм функции:

1. Для каждой ошибки

1.1 Сгенерировать сообщение об ошибке

4. Диаграмма вызовов функций приведена в приложении №1, UML-диаграмма в приложении №3

5. Диаграмма потоков данных приведена в приложении №2.

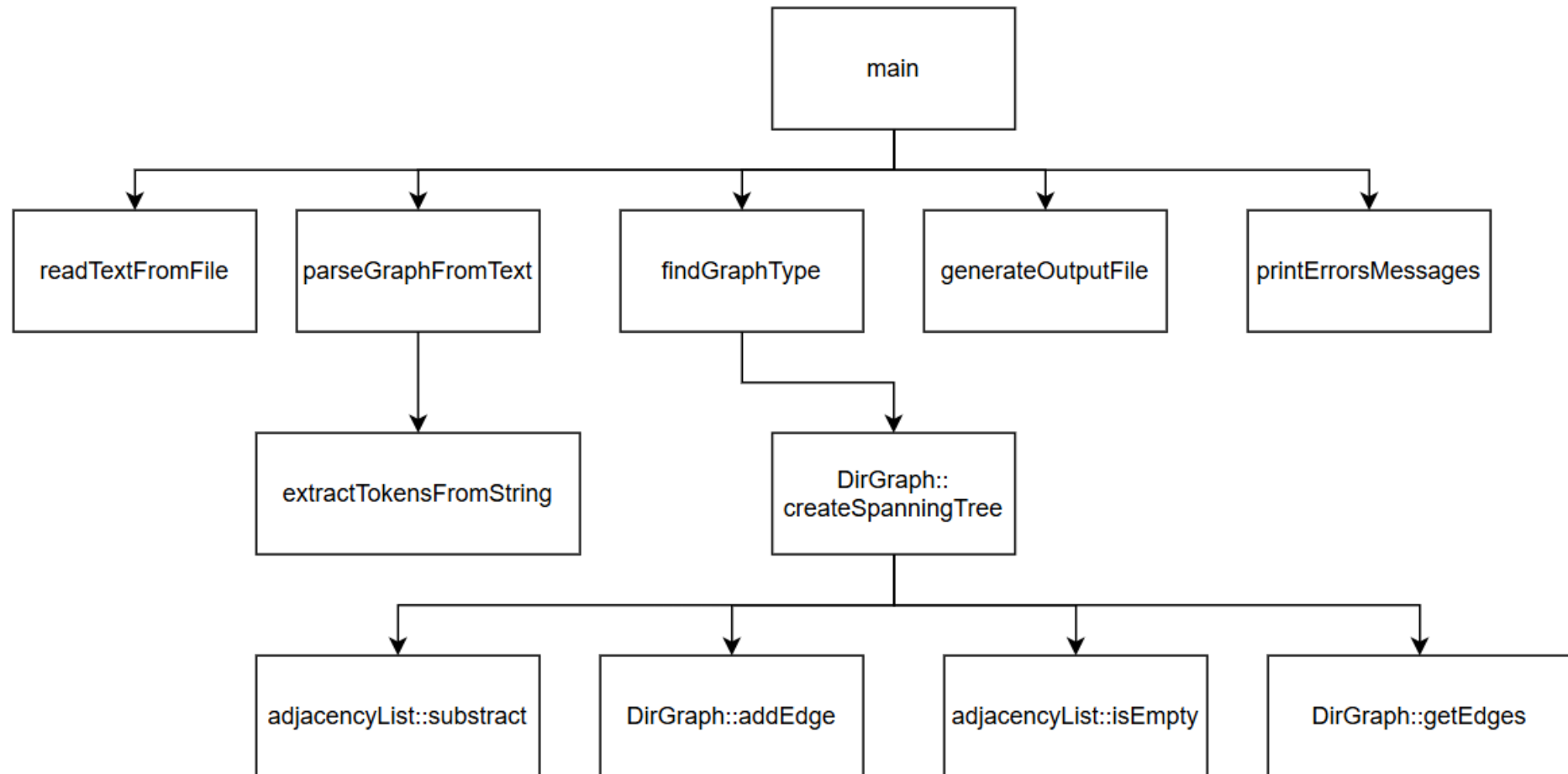


Рис 1. Диаграмма вызовов функций

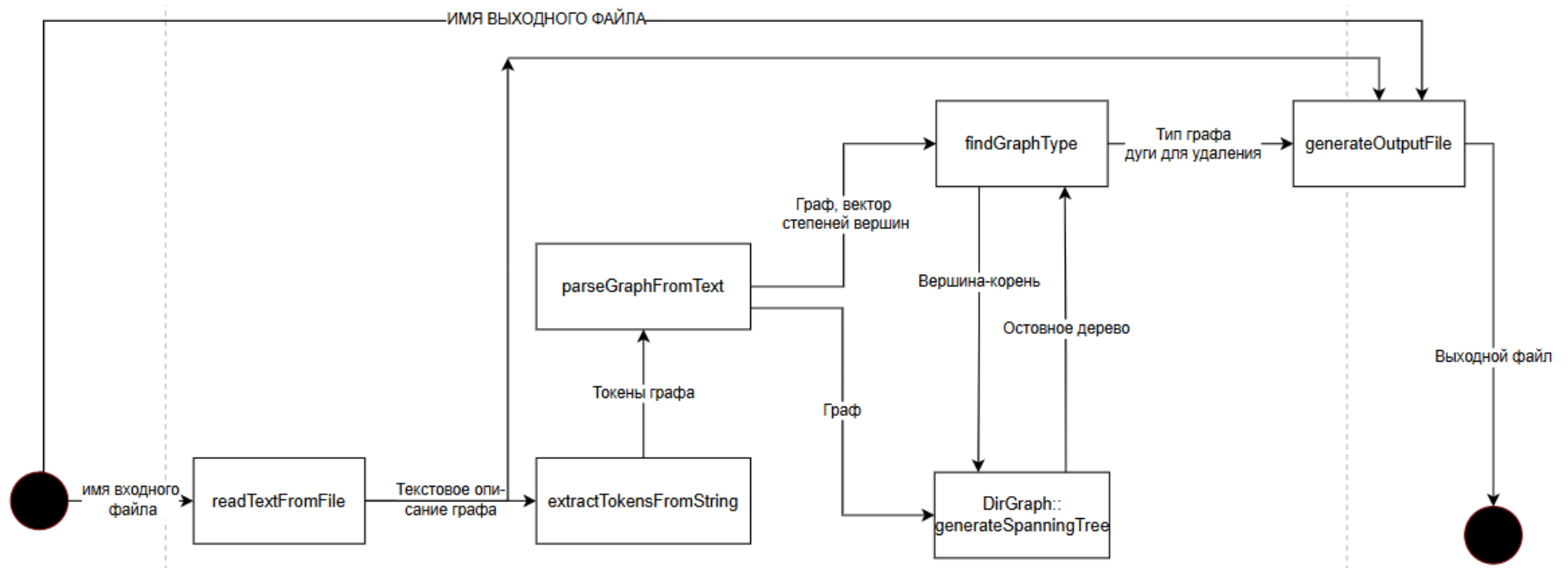


Рис 2. Диаграмма потоков данных

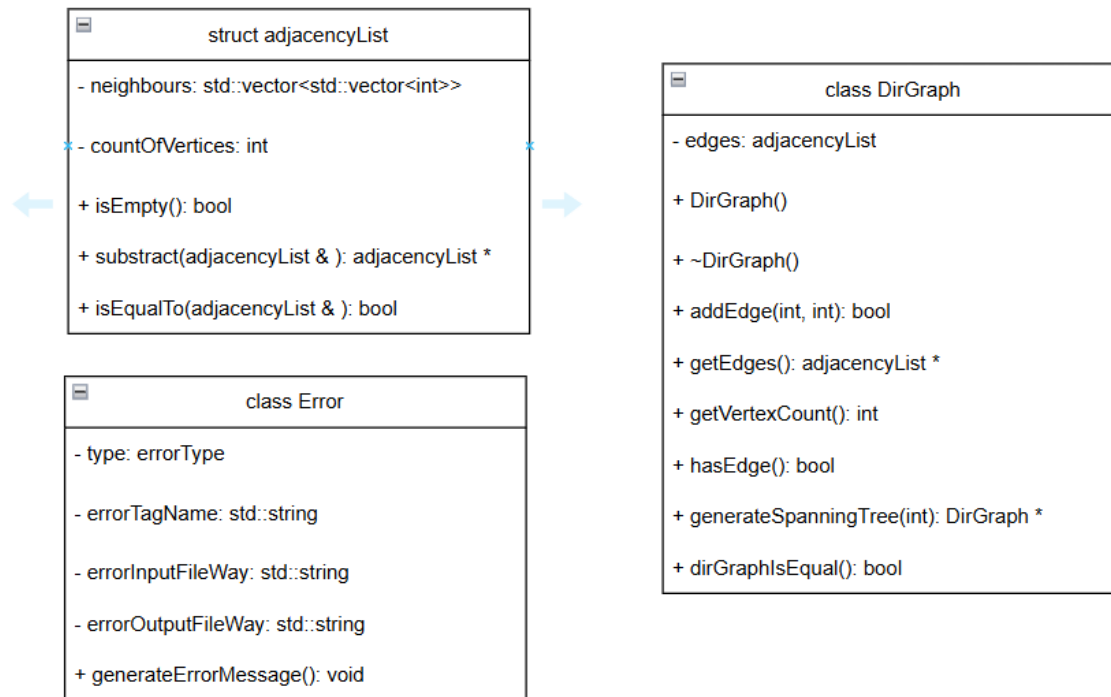


Рис 3. UML-диаграмма